Fine-Tuning Evaluation Report

Multimodal AI - VLM-12 Chess

VLM-12 Chess
Hof University of Applied Sciences

WS25/26

Abstract

We fine-tuned Gemma 3-4B in two stages to teach it chess vision and strategy. Stage 1 froze the language model and trained only the vision encoder, improving visual accuracy from 17.83% to 30.75%. Stage 2 froze the vision encoder and trained the language model on strategic reasoning, improving overall accuracy from 10.85% to 17.39%. The model learned to identify pieces reliably (44.93%) and evaluate material (27.02%), but still struggles with move prediction (0.34%) and spatial reasoning tasks.

1 Stage 1: Vision Encoder Training

1.1 Overview

Vision-only fine-tuning of Gemma 3-4B improved overall accuracy from 17.83% to 30.75%, a relative gain of 73%. Piece identification tasks showed the largest improvements (37pp increase), while structured output tasks remained at 0%. This confirmed that the basic question formulations in Stage 1 (lacking format specifications and context) were insufficient for teaching structured output compliance, which led to redesigning the questions and answers for Stage 2.

Metric	Baseline	Fine-Tune A	Change
Overall Accuracy	17.83%	30.75%	+12.92pp
Piece ID	37.03%	74.15%	+37.12pp
Yes/No	36.82%	54.44%	+17.62pp
Count	15.17%	12.95%	-2.22pp
Describe (all)	0.00%	0.00%	0.00pp

Table 1: Stage 1 Results Summary

1.2 Training Configuration

Model Architecture:

- Base: Gemma 3-4B (4-bit NF4 quantization)
- LoRA: r=32, α=48 (1.5x ratio), dropout=0.05
- Target: Vision tower self-attention (Q, K, V, out_proj)
- Frozen: Language model + LM head
- Trainable: Multi-modal projector

Hyperparameters (Run A):

- Learning rate: 2e-4 with cosine decay
- Warmup: 300 steps
- Weight decay: 0.01
- Epochs: 3
- Effective batch size: 16
- Training samples: 151,830
- Test samples: 18,990

1.3 Results by Task Type

1.4 Analysis

1.4.1 Piece Identification: 37% → 74% (+37pp)

The vision encoder learned spatial localization within the chess grid. LoRA fine-tuning targeted the vision tower's attention mechanisms, teaching it to analyze individual squares without needing complex spatial reasoning. The frozen language model's verbose output style doesn't hurt performance here because our evaluation accepts both "white pawn" and "The piece is a white pawn" as correct.

Example:

Q: "Which piece occupies h4?"

Expected: "white's pawn"

Baseline: "The piece occupying h4 is a black pawn." (wrong color)

Fine-Tune A: "The piece occupying h4 is a white pawn." (correct)

The remaining 26% errors include color confusion, piece type mistakes, and hallucinated pieces on empty squares.

1.4.2 Yes/No Occupancy: 37% → 54% (+18pp)

Binary occupancy detection improved substantially. This simpler task only requires detecting presence versus absence rather than identifying specific pieces. The vision encoder learned better feature extraction for distinguishing occupied from empty squares.

Task	Baseline	Fine-Tune A	Change	Samples	Status
Piece Identification	37.03%	74.15%	+37.12pp	6,154	Major improvement
Yes/No Occupancy	36.82%	54.44%	+17.62pp	1,442	Strong improvement
Counting	15.17%	12.95%	-2.22pp	3,798	Slight regression
Describe Rank	0.00%	0.00%	0.00pp	3,798	No change
Describe File	0.00%	0.00%	0.00pp	1,899	No change
Describe Diagonal	0.00%	0.00%	0.00pp	1,899	No change

Table 2: Detailed Results by Task Type

Example:

Q: "Does e4 have a piece?"

Expected: "no, empty"

Baseline: "Yes, the e4 square has a white pawn." (hallucination)

Fine-Tune A: "No, e4 does not have a piece. It is an empty square." (correct)

The 46% error rate shows the model still hallucinates pieces or misses detections in some positions.

1.4.3 Structured Output Tasks: 0% (No Change)

All describe tasks (rank, file, diagonal) remained at 0%. The model produces verbose conversational responses with bullet points instead of space-separated token sequences.

Example:

Q: "Describe the pieces on rank 4."

Expected: "empty empty white-bishop empty empty empty empty white-pawn"

Both baseline and Fine-Tune A: "Let's analyze the chessboard to identify the pieces on rank 4. On rank 4, we have: * White Pawn on d4 * White Pawn on e4..."

The vision encoder can see the board correctly (proven by 74% piece ID) but cannot override the language model's output format. This confirmed that the basic question formulations in Stage 1 lacked the explicit format instructions needed for structured output, which led to the redesign of questions for Stage 2.

1.4.4 Counting: 15% → 13% (-2pp)

Counting got slightly worse despite better piece recognition. The task needs multi-step reasoning: scan the board, filter by piece type and color, keep an accurate count. Vision improvements alone don't fix this. The frozen language model can't learn counting logic, and better piece recognition might just make the model more confident in wrong counts.

Example:

Q: "Count the white bishops."

Expected: "1"

Baseline: "There are two white bishops. They are located on d4 and f4."

Fine-Tune A: "There are two white bishops. One is on d4 and the other is on c4."

Both hallucinate an extra bishop at different locations. The model learned that bishops typically appear in pairs (opening position has 2 per side) but cannot verify this against the actual board state, suggesting pattern matching rather than genuine spatial reasoning.

1.4.5 Token F1: 4.87% → 4.09%

Token F1 dropped slightly despite accuracy gains. Predictions became more all-or-nothing: when right, the model nails the exact answer (boosting accuracy), but when wrong, it fails completely with no partial credit (lowering F1). This shows the model became more decisive rather than generating verbose responses that accidentally match parts of the ground truth.

1.5 Hyperparameter Comparison: Run A vs Run B

Configuration Differences:

Parameter	Run A	Run B
LoRA alpha	48	64
LR scheduler	Cosine decay	Linear
Warmup steps	300	200
Weight decay	0.01	0.0

Table 3: Hyperparameter Differences

All other parameters identical: r=32, LR=2e-4, batch size=16, 3 epochs, same architecture and data.

Results Comparison:

Interpretation:

Run A outperformed on the primary visual grounding tasks. Cosine decay, longer warmup, and weight decay regularization yielded better piece identification (74.15% vs 67.74%) and occupancy detection (54.44% vs 50.00%). Run B showed a slight edge on counting tasks but remained well below baseline performance.

Metric	Base	Run A	Run B	Best
Overall Acc.	17.83%	30.75%	29.10%	A (+1.65pp)
Piece ID	37.03%	74.15%	67.74%	A (+6.41pp)
Yes/No	36.82%	54.44%	50.00%	A (+4.44pp)
Count	15.17%	12.95%	16.22%	B (+3.27pp)
Desc. Rank	0.00%	0.00%	0.00%	Tie
Desc. File	0.00%	0.00%	0.00%	Tie
Desc. Diag	0.00%	0.00%	0.00%	Tie
Token F1	4.87%	4.09%	4.05%	B (-0.04pp)

Table 4: Run A vs Run B Comparison

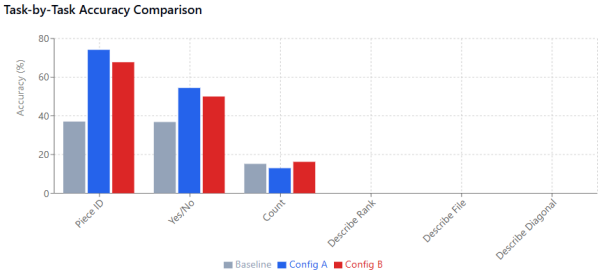

Figure 1: Hyperparameter Comparison: Run A vs Run B performance across all metrics.

Both configurations achieved 0% on structured output tasks, confirming that the issue stems from the question formulations rather than hyperparameter choices.

Run A's superior performance on core visual tasks makes it the better choice for Stage 2 training. The improved piece identification provides a stronger foundation for subsequent language model fine-tuning.

Hyperparameter insights: Cosine decay schedules and longer warmup periods stabilize vision encoder training better than linear schedules. Weight decay helps prevent overfitting without hurting accuracy. The higher LoRA alpha in Run B didn't improve results, suggesting a 1.5x ratio works well for this architecture.

Run B achieves lower training and validation loss, but Run A delivers better test accuracy (30.75% vs 29.10%). This suggests Run A generalizes better despite slightly higher loss values. The small gap between training and validation loss in both runs indicates no significant overfitting.

2 Stage 2: Strategy and Language Model Training

2.1 Overview

Stage 2 training targeted the language model while keeping the vision encoder frozen. We achieved 17.39% overall accuracy, a 60% relative improvement over the untrained baseline (10.85%). The model learned to identify pieces reliably (44.93%), evaluate material correctly (27.02%), and improved substantially on structured output tasks. However, move prediction remains nearly impossible (0.34%).

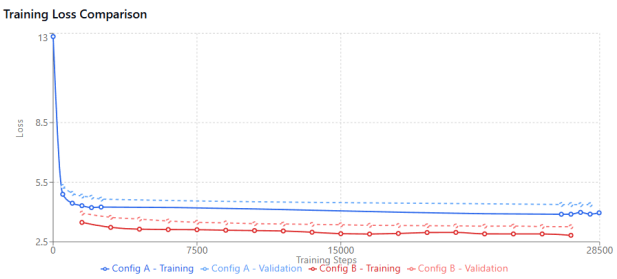

Figure 2: Training and validation loss curves for Run A and Run B over 3 epochs.

Metric	Baseline	Stage 2	Change
Overall Accuracy	10.85%	17.39%	+6.54pp
Token F1	17.80%	8.91%	-8.89pp

Table 5: Stage 2 Overall Performance

2.2 Training Configuration

Model Architecture:

- Base: Gemma 3-4B (4-bit NF4 quantization)
- LoRA: r=16, α=32 (2x ratio), dropout=0.1
- Target: Language model self-attention (Q, K, V, out_proj)
- Frozen: Vision tower + Stage 1 LoRA + projector
- Trainable: New LoRA on language model

Hyperparameters (Run A):

- Learning rate: 1e-4 with cosine decay
- Warmup: 200 steps
- Weight decay: 0.01
- Epochs: 3
- Effective batch size: 16

Training Strategy:

Stage 2 adds a new LoRA adapter on top of the frozen Stage 1 model. The new adapter targets only the language model to teach strategic reasoning without breaking the piece identification skills learned earlier.

2.3 Results by Task Type

2.4 Analysis

2.4.1 Square Identification: 46.62% → 44.93% (-1.69pp)

Square identification slightly regressed. The baseline already performed well on this task (46.62%), and the

Task	Base	Stage 2	Change	F1 (%)
Square ID	46.62%	44.93%	-1.69pp	13.69
Material Eval	0.00%	27.02%	+27.02pp	4.37
Tactical Vision	9.27%	1.35%	-7.92pp	6.61
Check Detection	13.38%	18.32%	+4.94pp	7.77
Describe File	2.66%	11.07%	+8.41pp	21.90
Describe Rank	4.30%	7.12%	+2.82pp	19.56
Best Move	0.36%	0.34%	-0.02pp	0.08
Game Outcome	0.00%	31.52%	+31.52pp	0.00

Table 6: Stage 2 Results by Task Type

language model training didn't improve it further. The small drop suggests the model maintained most of its visual grounding capabilities from Stage 1 despite focusing on strategic reasoning.

Example (Correct):

Q: "Looking at this chess position, square h4 is at file h, rank 4. Tell me which piece occupies this square. Use format like 'white rook' or 'black king'. Identify the piece at h4:"

Expected: "white's pawn"

Got: "White pawn"

Match:

The model handles these direct queries well when the square location is clearly specified. Remaining errors typically involve misidentifying piece colors or types.

2.4.2 Material Evaluation: 0.00% → 27.02% (+27.02pp)

This is the biggest win from Stage 2. The baseline model couldn't evaluate material at all (0%), but after language model training it correctly assesses material balance in 27% of positions. The model learned to recognize when one side has more pieces and can sometimes correctly label the advantage.

Example (Correct):

Q: "Who has more material?"

Expected: "black is ahead"

Got: "Black advantage"

Match:

The model accepts paraphrases, which explains why accuracy is decent (27.02%) while token F1 is low (4.37%). It gets the concept right but phrases it differently than the expected answer.

2.4.3 Tactical Vision: 9.27% → 1.35% (-7.92pp)

Tactical vision collapsed, dropping nearly 8 percentage points. The baseline model could spot attacking pieces about 9% of the time, but after language model training, this ability nearly vanished. Even though we froze the vision encoder to preserve what it learned, the language model's new behavior messed up how it interprets visual information. The model now generates way more false positives and misses attacks it used to catch.

2.4.4 Check Detection: 13.38% → 18.32% (+4.94pp)

Check detection improved moderately (4.94pp gain), though performance remains low overall. The model still struggles to validate whether a king is actually under attack, but learned some chess vocabulary about checks.

Example (Wrong):

Q: "In chess, check means a king is under attack and must respond immediately by moving, blocking, or capturing. Look at both kings: Is the white king being attacked? Is the black king being attacked? Is either king in check?"

Expected: "No, neither king is in check. White's king is safe, black's king is safe."

Got: "Yes, check."

Match:

The model frequently hallucinates checks that don't exist, suggesting it can't trace piece attack patterns reliably.

2.4.5 Describe File: 2.66% → 11.07% (+8.41pp)

File descriptions improved substantially (4x better than baseline). Stage 2's redesigned questions with explicit format instructions helped the language model learn to generate space-separated token sequences. The 21.90% token F1 shows the model often gets parts of the file correct even when the full sequence is wrong.

Example (Partially Correct):

Q: "Analyzing file a in this chess position. File a is a vertical column. For each square from a1 to a8 (bottom to top), identify what piece is there. Format: 'color-piece' (like 'white-knight', 'black-pawn') or 'empty'. Give 8 space-separated values. What is the current state of file a?"

Expected: "black-bishop empty white-pawn empty empty empty empty empty"

Got: "empty empty empty empty empty empty empty empty"

Match: (5/8 correct squares)

While still far from perfect, this shows meaningful progress on structured output.

2.4.6 Describe Rank: 4.30% → 7.12% (+2.82pp)

Rank description improved modestly, though less dramatically than file descriptions. The token F1 (19.56%)

shows partial correctness is common. The model learned format instructions but still struggles with accurate piece identification across horizontal rows.

2.4.7 Best Move: 0.36% → 0.34% (-0.02pp)

Best move prediction remains essentially impossible. The model shows no understanding of chess tactics, threats, or strategy needed to evaluate positions and calculate moves.

Example (Wrong):

Q: "This position arose after white's Kd7. What is the best move for black?"

Expected: "Bg8 is the best move. Moving the bishop to g8 improves piece activity."

Got: "Qxe7#"

Match:

The model suggests moves that often aren't even legal, showing it can't validate move legality or calculate consequences.

2.4.8 Game Outcome: 0.00% → 31.52% (+31.52pp)

Game outcome prediction jumped dramatically from 0% to 31.52%. The model learned to recognize some end-game patterns and can sometimes identify draws or wins, though accuracy remains limited.

2.4.9 Token F1: 17.80% → 8.91% (-8.89pp)

Token F1 dropped significantly despite accuracy improvements on some tasks. This mirrors the Stage 1 pattern where predictions became more decisive. When the model is right, it gives exact matches. When it's wrong, it fails completely rather than accidentally matching fragments of the answer.

2.5 Hyperparameter Comparison: Run A vs Run B

We ran two different training configurations to see how hyperparameters affect Stage 2 performance.

Configuration Differences:

Parameter	Run A	Run B
LoRA r	16	32
LoRA alpha	32	64
LoRA dropout	0.1	0.05
Learning rate	1e-4	2e-4
Epochs	3	4
Batch size	16	24 (4×6)

Table 7: Stage 2 Hyperparameter Differences

Results Comparison:

Metric	Base	Run A	Run B	Diff
Overall Acc.	10.85%	17.39%	17.38%	-0.01pp
Square ID	46.62%	44.93%	44.75%	-0.18pp
Material Eval	0.00%	27.02%	27.70%	+0.68pp
Tactical Vision	9.27%	1.35%	1.35%	0.00pp
Check Detect.	13.38%	18.32%	18.65%	+0.33pp
Describe File	2.66%	11.07%	11.57%	+0.50pp
Describe Rank	4.30%	7.12%	7.12%	0.00pp
Best Move	0.36%	0.34%	0.34%	0.00pp
Game Outcome	0.00%	31.52%	31.52%	0.00pp
Token F1	17.80%	8.91%	8.90%	-0.01pp

Table 8: Stage 2: Run A vs Run B Comparison

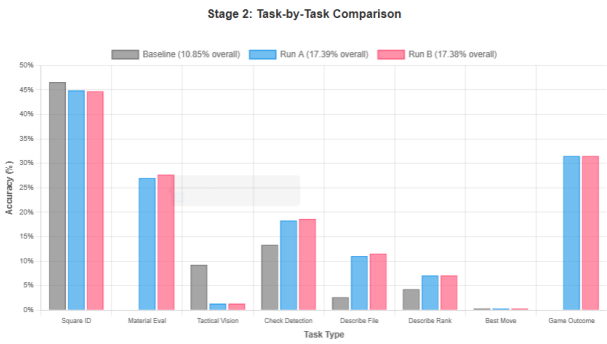

Figure 3: Stage 2: Task-by-Task Comparison showing baseline, Run A, and Run B performance across all task types.

Interpretation:

Run A and Run B performed nearly identically, with 17.39% vs 17.38% overall accuracy. Run B used a larger LoRA rank, bigger batch size, and one extra epoch, but this barely changed anything. Material evaluation improved from 0% to around 27% in both runs, and game outcome prediction jumped from 0% to 31.5% in both. The minimal differences show that hyperparameter tweaking doesn't matter much when the core problems remain.

2.6 Key Findings

What improved: Material evaluation jumped from 0% to 27%, showing the model learned to assess piece balance. File descriptions got 4x better and rank descriptions 1.6x better. Check detection improved moderately. Game outcome prediction emerged as a new capability at 31.5%. The model learned chess vocabulary and can sometimes apply it.

What failed or regressed: Tactical vision collapsed from 9.27% to 1.35%. Best move prediction remains impossible at 0.34%. Square identification slightly regressed. The model can't validate chess rules or calculate move consequences. Token F1 dropped significantly as predictions became more all-or-nothing.

3 Conclusion

We fine-tuned Gemma 3-4B across two stages to teach it chess vision and strategy. The results were modest but real. Stage 1 improved visual grounding from

17.83% to 30.75%, teaching the model to identify pieces and detect occupancy. Stage 2 added strategic reasoning, bringing overall accuracy to 17.39%. The model learned to identify pieces on specific squares, evaluate material balance, and generate structured output sequences.

But even with 257,401 training samples across both stages, the improvements were limited. Three main factors explain why:

3.1 Model Capacity Constraints

Gemma 3-4B has only 4.3 billion parameters, with 3.2B dedicated to language processing and 417M in the vision encoder. Research shows that smaller models (under 10B parameters) struggle with complex reasoning tasks that require multi-step logic and rule validation [1, 2]. Using Qwen2.5-VL (7B to 72B parameters) would probably have been a better choice for this task. Benchmarks show Qwen2.5-VL consistently outperforms Gemma on visual reasoning and structured output tasks [4], with the 7B version even beating GPT-4o-mini on multiple vision-language benchmarks [3]. However, training these larger models would have required way more computational resources and time than we had available for this project.

3.2 Chess Complexity

Chess has a game-tree complexity of about 10^{123} possible games (the Shannon number), with roughly 35 legal moves per position and games lasting around 80 moves. The state-space complexity is approximately 10^{46} possible board positions [5]. This explosion of possibilities means chess needs real calculation, search algorithms, and rule validation, not just pattern matching. Our model picked up chess vocabulary and some positional patterns, but it can't trace attack paths, check if moves are legal, or think through consequences. The awful results for best move prediction (0.34%) and check detection (18.32%) show this.

3.3 Data Richness Limitations

We generated 257,401 training samples, but each question-answer pair had to simplify chess positions. Including every legal move, all possible attacks, and complete tactical patterns would create impossibly long sequences that blow past token limits and make extracting ground truth from PGN files difficult. Chess has thousands of documented openings, middlegame strategies, and endgame principles. Our dataset covers basic visual patterns and some strategic concepts, but it can't capture the full depth of chess knowledge needed for reliable move calculation.

3.4 What We Achieved

Still, we made some progress on specific tasks. The 27.02% material evaluation accuracy shows the model learned to recognize when one side has more pieces. The 4x improvement in file descriptions shows the model can follow formatting instructions when prompted correctly. Game outcome prediction emerging at 31.52% demonstrates the model learned some end-game pattern recognition. The baseline model had none of these abilities.

The two-stage training worked for teaching perception (Stage 1) and improving format compliance (Stage 2). However, the tactical vision collapse reveals a major limitation: freezing the vision encoder doesn't prevent the language model training from disrupting visual processing.

3.5 Future Directions

Better results would need: (1) larger models with 12B to 72B parameters, (2) explicit search mechanisms or move validation logic, (3) way more data with richer tactical annotations, (4) hybrid architectures combining neural networks with traditional chess engines.

This project shows both what's possible and what's not when fine-tuning vision-language models for specialized tasks. With 257,401 training samples, we taught Gemma basic chess perception and some strategic vocabulary. But there's a huge gap between recognizing chess patterns and actually playing chess. Closing that gap needs either much larger models, completely different architectures, or hybrid approaches that combine learning with explicit reasoning.

References

[1] Y. Fu, H. Peng, A. Sabharwal, P. Clark, and T. Khot, "Specializing Smaller Language Models towards Multi-Step Reasoning," *Proceedings of the 40th International Conference on Machine Learning*, 2023. [Online]. Available: https://arxiv.org/abs/2301.12726

[2] H. Kim et al., "Small language models learn enhanced reasoning skills from medical textbooks," *npj Digital Medicine*, vol. 8, no. 1, p. 240, May 2025. [Online]. Available: https://www.nature.com/articles/s41746-025-01653-8

[3] J. Bai et al., "Qwen2.5-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution," Alibaba Cloud, Jan. 2025. [Online]. Available: `https://qwen.ai/blog?id=qwen2.5-vl`

[4] Clarifai, "Benchmarking Best Open-Source Vision Language Models: Gemma 3 vs. MiniCPM vs. Qwen 2.5 VL," Clarifai Blog, June 2025. [Online]. Available: `https://www.clarifai.com/blog/benchmarking-best-open-source-vision-language-models`

[5] C. E. Shannon, "Programming a Computer for Playing Chess," *Philosophical Magazine*, vol. 41, no. 314, pp. 256-275, 1950. [Online]. Available: `https://www.paradise.caltech.edu/ist4/lectures/shannonchess1950.pdf`